Magnitude Comparator, Decoder and Multiplexer

2023/11/06

B113040045 許育菖

實驗內容

Exercise1

- Design and verify the 4-bit magnitude comparator by using **Verilog Structural level modeling.**
- Algorithm -> logic

  $A = A_3A_2A_1A_0$; $B = B_3B_2B_1B_0$

  $A = B$ if $A_3=B_3$, $A_2=B_2$, $A_1=B_1$ and $A_1=B_1$

  equality: $x_i = A_iB_i + A_i'B_i'$

  $(A=B) = x_3x_2x_1x_0$

  $(A>B) = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$

  $(A<B) = A_3'B_3 + x_3A_2'B_2 + x_3x_2A_1'B_1 + x_3x_2x_1A_0'B_0$

- Implementation

  xi = (AiBi' +Ai'Bi)'

Algorithm -> logic

$A = A_3A_2A_1A_0$

$B = B_3B_2B_1B_0$

A=B: if E = 1 1010, 1010

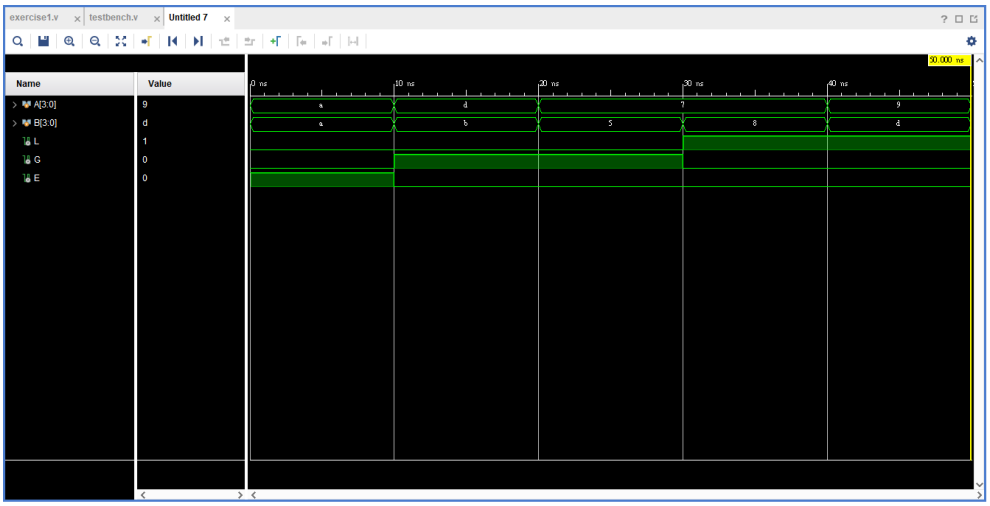
A>B: if G = 1 1101, 1011 and 0111, 0101

A<B: if L = 1 0111, 1000 and 1001, 1101

Verilog code

```
`timescale 1ns / 1ps

module exercise1(
    output L, G, E,
    input [3:0] A, [3:0] B
);

    //  =
    wire n0, n1, n2, n3;
    or(n0, (A[0] & B[0]), ( (!A[0]) & (!B[0]) ) );
    or(n1, (A[1] & B[1]), ( (!A[1]) & (!B[1]) ) );
    or(n2, (A[2] & B[2]), ( (!A[2]) & (!B[2]) ) );
    or(n3, (A[3] & B[3]), ( (!A[3]) & (!B[3]) ) );

    and(E, n0, n1, n2, n3);

    //  >
    wire n4, n5, n6, n7;
    and(n4, A[3], (!B[3]));
    and(n5, A[2], (!B[2]), n3);
    and(n6, A[1], (!B[1]), n3, n2);
    and(n7, A[0], (!B[0]), n3, n2, n1);

    or(G, n4, n5, n6, n7);

    //  <
```

```
    wire n8, n9, n10, n11;
    and(n8, (!A[3]), B[3]);
    and(n9, (!A[2]), B[2], n3);
    and(n10, (!A[1]), B[1], n3, n2);
    and(n11, (!A[0]), B[0], n3, n2, n1);

    or(L, n8, n9, n10, n11);

endmodule
```



Exercise2

Design and verify a 3-to-8 decoder composed of two 2-to-4 decoders.

- Dataflow modeling
- Structural level (Gate-level) modeling

Verilog code

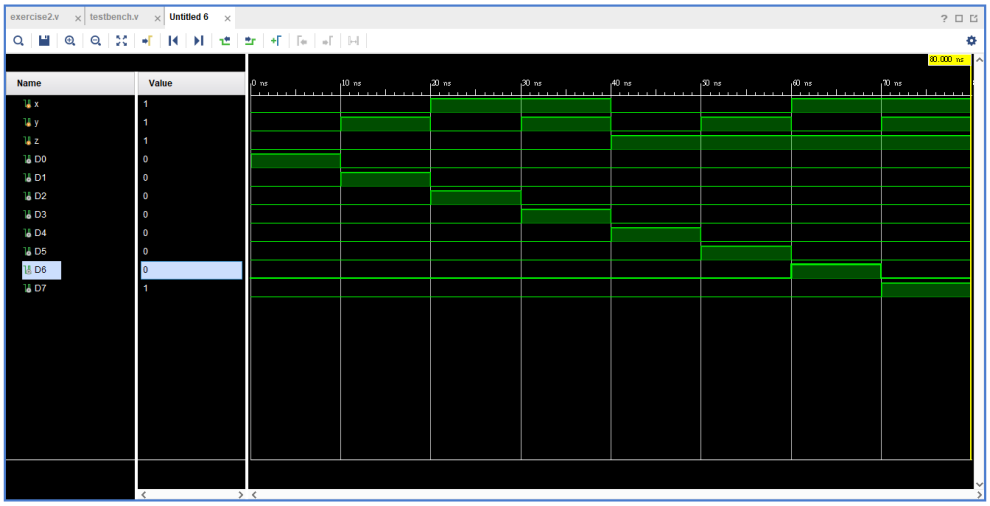
```
`timescale 1ns / 1ps

module exercise2(
    output D0, D1, D2, D3, D4, D5, D6, D7,
    input x, y, z
    );
```

```
decoder A(D0, D1, D2, D3, x, y, (!z));

decoder B(D4, D5, D6, D7, x, y, z);

endmodule
```

The Decoder

```
`timescale 1ns / 1ps

module decoder(D0, D1, D2, D3, A, B, E);
        output D0, D1, D2, D3;
        input A, B, E;

and(D0, (!A), (!B), E);
and(D1, (!A), B, E);
and(D2, A, (!B), E);
and(D3, A, B, E);

endmodule
```

先建立兩個 2-to-4 decoder，接著再把兩個 2-to-4 decoder 合併形成 3-to-8 decoder。



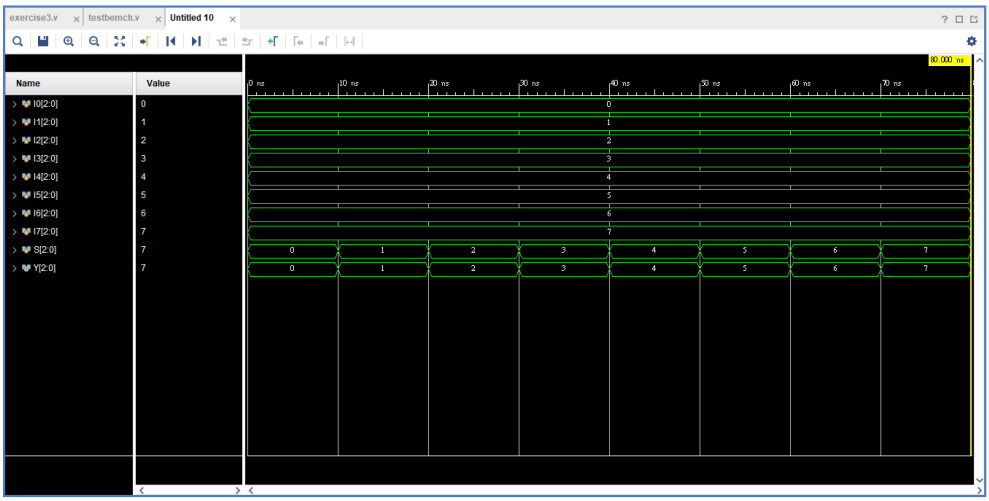
Exercise3

Design and verify the 8-to-1 multiplexer using Verilog HDL

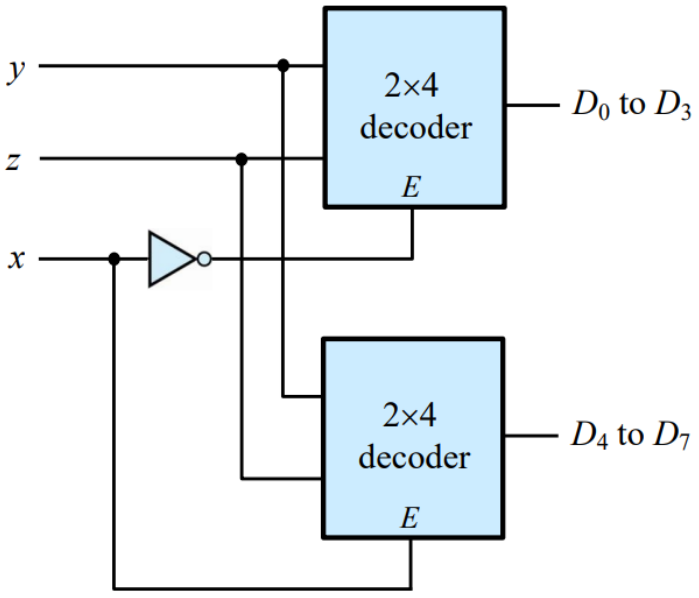
- Behavioral level modeling
- Dataflow modeling
- Structural level (Gate-level) modeling

Verilog code

```
`timescale 1ns / 1ps

module exercise3(
        output reg [2:0] Y,
        input [2:0]I0, I1, I2, I3, I4, I5, I6, I7,
        input [2:0] S
);
        always @ (I0, I1, I2, I3, I4, I5, I6, I7, S)
                case (S)
                        3'b000:         Y = I0;
                        3'b001:         Y = I1;
                        3'b010:         Y = I2;
                        3'b011:         Y = I3;
                        3'b100:         Y = I4;
                        3'b101:         Y = I5;
                        3'b110:         Y = I6;
                        3'b111:         Y = I7;
                endcase
endmodule
```



實驗心得

此次實驗是我們首次使用 Behavioral level modeling 來撰寫 verilog，我原本以為新的 modeling 會不太好寫，為此還特地從第三題開始進行實驗，沒想到比想像中的還簡單，三題總共只花費一個多小時就完成了，讓我難得地提早交卷。

這次實驗我在撰寫第二題的時候發生了一個小插曲，題目圖片上的 x, y, z 沒有按照順序排，我以為是出題者不小心打錯了所有就把 x, y, z 按順序輸入，沒想到輸出出來的圖形雖然是正確的，但排序完全錯誤，看來我錯怪出題者了，sorry。